REST API 설계

1. 인증 (Authentication) API

기능	HTTP Method	URL	요청 Body (JSON)	응답 (성공 시)
회원 가입	POST	/api/auth/signup	email , password , nickname , region	200 OK - TokenResponse
로그인	POST	/api/auth/login	email , password	200 OK - TokenResponse
로그아웃	POST	/api/auth/logout	-	200 OK
토큰 재발급	POST	/api/auth/refresh	-	200 OK - TokenResponse

2. 회원 (User) API

기능	HTTP Method	URL	요청 Body (JSON)	응답 (성공 시)
내 정보 조회	GET	/api/users/me	-	200 OK - UserDetailRespo
회원 상세 조회	GET	/api/users/{userId}	-	200 OK - UserDetailRespo
내 정보 수정	PATCH	/api/users/me	nickname , region , introduction , profileImageUrl	200 OK - ApiResponse
회원 탈퇴	DELETE	/api/users/me	-	200 OK - ApiResponse
프로필 이미지 업로드/수정	POST	/api/users/{userId}/profile-image	multipart/form-data	200 OK - 업로드된 이미지 경
프로필 이미지 삭제	DELETE	/api/users/{userId}/profile-image	-	200 OK - ApiResponse
내가 가입한 그룹 목록 조회	GET	/api/groups/my	-	200 OK - 가입한 그룹 목록 (List<GroupResponse>)
내가 찜한 그룹 목록 조회	GET	/api/my/liked-groups	-	200 OK - 찜한 그룹 목록 (PagingResponse<GroupR
내가 작성한 게시글 목록 조회	GET	/api/my/posts	-	200 OK - 작성한 게시글 목록 (PagingResponse<PostRes
내가 작성한 댓글 목록 조회	GET	/api/my/comments	-	200 OK - 작성한 댓글 목록 (PagingResponse<MyCom
내가 참석하는 이벤트 목록 조회	GET	/api/my/upcoming-events	-	200 OK - 다가오는 이벤트 목 (List<EventResponse>)

3. 그룹 (Group) API

기능	HTTP Method	URL	요청 Body (JSON)
그룹 생성	POST	/api/groups	name , description , region , maxMembers , imageUrl , tags , catego
그룹 검색 및 조회	GET	/api/groups	-
그룹 상세 조회	GET	/api/groups/{groupId}	-
그룹 정보 수정	PATCH	/api/groups/{groupId}	name , description , region , maxMembers , tags , categoryId
그룹 삭제	DELETE	/api/groups/{groupId}	-
그룹 커버 이미지 업로드/수정	POST	/api/groups/{groupId}/cover-image	multipart/form-data
그룹 커버 이미지 삭제	DELETE	/api/groups/{groupId}/cover-image	-
모임장 위임	PATCH	/api/groups/{groupId}/delegate-owner/{newOwnerId}	-

그룹 찜하기/찜 취소	POST	/api/groups/{groupId}/like	-
그룹 가입 신청	POST	/api/groups/{groupId}/join	-
그룹 탈퇴	DELETE	/api/groups/{groupId}/leave	-
그룹 멤버 목록 조회	GET	/api/groups/{groupId}/members	-
그룹 멤버 강퇴 (모임장)	DELETE	/api/groups/{groupId}/members/{userId}	-

4. 게시글 (Post) API

기능	HTTP Method	URL	요청 Body (JSON)	응답 (성공 시)
게시글 작성	POST	/api/groups/{groupId}/posts	title, content, imageUrl	201 Created - 생성된 게시글 정보 (L Header)
게시글 목록 조회	GET	/api/groups/{groupId}/posts	-	200 OK - 해당 그룹의 게시글 목록 (PagingResponse<PostResponse
게시글 상세 조회	GET	/api/groups/{groupId}/posts/{postId}	-	200 OK - 특정 게시글 상세 정보 (PostDetailResponse)
게시글 수정	PATCH	/api/groups/{groupId}/posts/{postId}	title, content, imageUrl	200 OK - ApiResponse
게시글 삭제	DELETE	/api/groups/{groupId}/posts/{postId}	-	200 OK - ApiResponse
게시글 이미지 업로드/수정	POST	/api/groups/{groupId}/posts/{postId}/image	multipart/form-data	200 OK - 업로드된 이미지 경로 (Stri
게시글 이미지 삭제	DELETE	/api/groups/{groupId}/posts/{postId}/image	-	200 OK - ApiResponse
게시글 무한스크롤 조회	GET	/api/groups/{groupId}/posts/infinite	-	200 OK - 커서 기반 게시글 목록 (CursorPagingResponse<PostRes
전체 게시글 검색	GET	/api/posts/search	-	200 OK - 검색된 게시글 목록 (PagingResponse<PostResponse

5. 댓글 (Comment) API

기능	HTTP Method	URL	요청 Body (JSON)	응답 (성공 시)
댓글/답글 작성	POST	/api/groups/{groupId}/posts/{postId}/comments	content, parentId (선택)	201 Created - 생성된 보 (Location Header)
댓글 목록 조회	GET	/api/groups/{groupId}/posts/{postId}/comments	-	200 OK - 해당 게시글의 목록 (List<CommentRespo
댓글 수정	PATCH	/api/groups/{groupId}/posts/{postId}/comments/{commentId}	content	200 OK - ApiRespons
댓글 삭제	DELETE	/api/groups/{groupId}/posts/{postId}/comments/{commentId}	-	200 OK - ApiRespons

6. 카테고리 (Category) API

기능	HTTP Method	URL	요청 Body (JSON)	응답 (성공 시)
카테고리 목록 조회	GET	/api/categories	-	200 OK - 전체 카테고리 목록 (List<CategoryResponse>)

7. 이벤트 (Event) API

기능	HTTP Method	URL	요청 Body (JSON)	응답 (성공
이벤트 생성	POST	/api/groups/{groupId}/events	title, description, location, imageUrl, eventDate, maxAttendees	201 Creat (Location
이벤트 목록 조회	GET	/api/groups/{groupId}/events	-	200 OK - (PagingR
이벤트 상세 조회	GET	/api/events/{eventId}	-	200 OK - (EventRe

이벤트 수정	PATCH	/api/events/{eventId}	title , description , location , imageUrl , eventDate , maxAttendees	200 OK - (EventRe
이벤트 삭제	DELETE	/api/events/{eventId}	-	200 OK -

8. 이벤트 참석 (Event Attendee) API

기능	HTTP Method	URL	요청 Body (JSON)	응답 (성공 시)	비고
참석 신청	POST	/api/events/{eventId}/attendees	-	200 OK - 성공 메시지 (String)	정원 미달 CONFIRI 원 초과 시 WAITING 등록
참석 취소	DELETE	/api/events/{eventId}/attendees/me	-	200 OK - ApiResponse	취소 시 다 가장 오래 자동 승격
참석자 목록 조회	GET	/api/events/{eventId}/attendees	-	200 OK - 해당 이벤트 참석자 목록 (List<EventAttendeeResponse>)	
참석 상태 변경	PATCH	/api/events/{eventId}/attendees/{userId}	{"status": "CONFIRMED"}	200 OK - 변경된 참석 정보 (EventAttendeeResponse)	

9. 이미지 업로드 전용 API

기능	HTTP Method	URL	요청 Body (JSON)	응답 (성공 시)
이미지 업로드	POST	/api/images	multipart/form-data	200 OK - 업로드된 이미지 URL (Map<String, String>)

10. 에러 코드 체계

HTTP Status	에러 코드	메시지
400 Bad Request	VALID_001	입력값이 올바르지 않습니다
401 Unauthorized	AUTH_001	아이디 또는 비밀번호가 올바르지 않습니다
403 Forbidden	AUTH_002	접근 권한이 없습니다
404 Not Found	GROUP_001	해당 그룹을 찾을 수 없습니다
404 Not Found	EVENT_001	해당 이벤트를 찾을 수 없습니다
409 Conflict	DUPLICATE_EMAIL	이미 존재하는 이메일입니다
409 Conflict	DUPLICATE_USERNAME	이미 존재하는 사용자명입니다
409 Conflict	GROUP_FULL	그룹 정원이 모두 찼습니다
409 Conflict	ALREADY_MEMBER	이미 그룹에 가입된 회원입니다
409 Conflict	EVENT_FULL	이벤트 정원이 모두 찼습니다
409 Conflict	EVENT_ALREADY_ATTENDING	이미 이벤트에 참석 신청이 되어 있습니다
500 Internal Server Error	SERVER_001	서버 내부 오류가 발생했습니다

public enum ErrorCode {

```
// 인증/인가 관련
INVALID_CREDENTIALS("AUTH_001", "아이디 또는 비밀번호가 올바르지 않습니다", HttpStatus.UNAUTHORIZED),
ACCESS_DENIED("AUTH_002", "접근 권한이 없습니다", HttpStatus.FORBIDDEN),

// 데이터 검증 관련
VALIDATION_ERROR("VALID_001", "입력값이 올바르지 않습니다", HttpStatus.BAD_REQUEST),
DUPLICATE_USERNAME("VALID_002", "이미 존재하는 사용자명입니다", HttpStatus.CONFLICT),

// 그룹/멤버십 관련
GROUP_NOT_FOUND("GROUP_001", "해당 그룹을 찾을 수 없습니다", HttpStatus.NOT_FOUND),
GROUP_FULL("GROUP_002", "그룹 정원이 모두 찼습니다", HttpStatus.CONFLICT),
ALREADY_MEMBER("GROUP_003", "이미 그룹에 가입된 회원입니다", HttpStatus.CONFLICT),
```

```
// 이벤트 관련
EVENT_NOT_FOUND("EVENT_001", "해당 이벤트를 찾을 수 없습니다", HttpStatus.NOT_FOUND),
EVENT_FULL("EVENT_002", "이벤트 정원이 모두 찼습니다", HttpStatus.CONFLICT),
EVENT_ALREADY_ATTENDING("EVENT_003", "이미 이벤트에 참석 신청이 되어 있습니다", HttpStatus.CONFLICT),

// 서버 오류
INTERNAL_SERVER_ERROR("SERVER_001", "서버 내부 오류가 발생했습니다", HttpStatus.INTERNAL_SERVER_ERROR);

private final String code;
private final String message;
private final HttpStatus status;

ErrorCode(String code, String message, HttpStatus status) {
    this.code = code;
    this.message = message;
    this.status = status;
}

public String getCode() { return code; }
public String getMessage() { return message; }
public HttpStatus getStatus() { return status; }
```

}